

AI Usage Log

ในการจัดทำโครงการระบบจัดการลานจอดรถด้วยภาษา Java ผู้จัดทำได้ใช้ AI เป็นเครื่องมือช่วยในการวิเคราะห์ ออกแบบ พัฒนา และตรวจสอบโปรแกรม โดยมี Prompt ที่ใช้ดังนี้

ครั้งที่	หัวข้อ	Prompt ที่ใช้
1	วิเคราะห์โจทย์	"ช่วยวิเคราะห์โจทย์ระบบลานจอดรถที่มีทางเข้าออกด้านเดียว โดยใช้ Stack และอธิบาย Input, Output, Constraints, Assumptions และ Edge Cases ให้น้อย"
2	ออกแบบโครงสร้างโปรแกรม	"ช่วยออกแบบโครงสร้างโปรแกรม Java สำหรับระบบลานจอดรถ โดยแบ่งเป็น Vehicle Class, ParkingRecord Class, Stack Manager และ Main Class"
3	Vehicle Class	"ช่วยเขียนและอธิบาย Vehicle Class ภาษา Java สำหรับเก็บ License Plate, Owner Name, Entry Time และ Vehicle Type"
4	ParkingRecord Class	"ช่วยออกแบบ ParkingRecord Class ภาษา Java สำหรับเก็บจำนวน Push, Pop, Comparison, Cars Moved และ Execution Time"
5	Stack Manager	"ช่วยออกแบบ StackManager ภาษา Java โดยใช้ parkingStack และ temporaryStack สำหรับจัดการรถในลานจอด"
6	Algorithm A	"ช่วยเขียน Algorithm A แบบ Iteration สำหรับค้นหาและนำรถเป้าหมายออกจาก Stack โดยใช้ parkingStack และ temporaryStack พร้อมอธิบายขั้นตอนการทำงาน"
7	Debug โปรแกรม	"โค้ด Java ของผมไม่สามารถ Run ใน VS Code ได้ ช่วยวิเคราะห์สาเหตุและแนะนำวิธีแก้ไขทีละขั้นตอน"

8	Input Validation	"ช่วยเพิ่ม Input Validation ให้โปรแกรม Java ระบบลานจอดรถ เช่น ตรวจสอบข้อมูลว่า ทะเบียนรถซ้ำ และตัวเลือก Menu ไม่ถูกต้อง"
9	Error Handling	"ช่วยออกแบบ Error Handling สำหรับระบบลานจอดรถ กรณี Stack ว่าง ไม่พบรถเป้าหมาย และผู้ใช้กรอกข้อมูลไม่ถูกต้อง"
10	ทดสอบโปรแกรม	"ช่วยตรวจสอบผลลัพธ์การทำงานของโปรแกรมจาก Output ที่ผมได้ โดยเฉพาะ Target Found, Cars Moved, Push, Pop และ Comparison ว่าถูกต้องหรือไม่"
11	Class Diagram	"ช่วยจัดทำ Class Diagram และอธิบายความสัมพันธ์ระหว่าง Main, Vehicle, ParkingRecord และ StackManager สำหรับโปรแกรมระบบลานจอดรถ"
12	จัดทำรายงาน	"ช่วยเรียบเรียงบทที่ 6 การพัฒนาโปรแกรม Java โดยมีหัวข้อ Class Diagram, โครงสร้างคลาส, เมธอดสำคัญ, Input Validation และ Error Handling"

การใช้ผลลัพธ์จาก AI

ผู้จัดทำใช้ AI เพื่อช่วยในการ ทำความเข้าใจโจทย์ ออกแบบโครงสร้างโปรแกรม ตรวจสอบแนวทางการเขียนโค้ด วิเคราะห์ข้อผิดพลาด และเรียบเรียงเอกสาร โดยผู้จัดทำเป็นผู้ตรวจสอบ แก้ไข และทดสอบโปรแกรมด้วยตนเองก่อนนำไปใช้งานจริง

วิธีแก้ไขและตรวจสอบ

1. เปรียบเทียบผลลัพธ์กับเอกสารประกอบการสอน

2. ตรวจสอบความสอดคล้องระหว่าง
 - Pseudocode
 - Source Code
 - Flowchart
3. ทดสอบโปรแกรมจริงด้วย Test Cases
4. วิเคราะห์ Time Complexity ด้วยตนเองอีกครั้ง

6 การวิเคราะห์ด้วยตนเอง

หลังจากได้รับคำแนะนำจาก AI สมาชิกในกลุ่มได้ร่วมกันวิเคราะห์และตรวจสอบความถูกต้องของอัลกอริทึมอีกครั้ง โดยพิจารณาจากหลักการทำงานของ Stack และผลการทดลองจริง เพื่อให้มั่นใจว่าผลลัพธ์ที่นำเสนอมีความถูกต้องและสอดคล้องกับเนื้อหา
รายวิชา